labwork 9

October 11, 2026

0.1 Histogram

We want to make an histogram of the intensities of an image.

0.1.1 Preparation

First, we need to transform our image with a grayscale, this will give us only one value corresponding to the intensity and make it easier to work with.

```
@cuda.jit
def grayscale(src, dst):
    x = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    y = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    if y >= src.shape[0] or x >= src.shape[1]:
        return
    g = numba.uint8((src[y, x, 0] + src[y, x, 1] + src[y, x, 2]) / 3)
    dst[y, x] = g
```

0.1.2 Reduction

The histogram kernel will transform an input image with the form of a 2D array in a array of size 256 containing the number of pixels that have the corresponding intensity.
We start by initializing an array containing zeros so if a value is not in the image its value is zero and not null

```
cuda.to_device(np.zeros((rangeNumber), int))
```

In order to count the number of pixels that have a certain intensity, each threads corresponding to each pixels have to add elements in the same list. The problem is that if many threads try to access the same element at the same time it will create a random result, all the values will not be counted. To avoid this problem, we use atomic operations. Thanks to that before writing to the arrays the threads will wait before the last thread is finished and not write at the same time but it means most of the time threads are just waiting. To optimize a little bit we use shared memory to make an histogram by block and then add them all together.

```
sharedHist = cuda.shared.array(256, np.int32)
sharedHist[tid] = 0

cuda.syncthreads()
cuda.atomic.add(sharedHist, src[y, x], 1)
cuda.syncthreads()

cuda.atomic.add(dst, tid, sharedHist[tid])
```

0.2 Equalize

We want to equalize our image using the histogram we just created. To do that we apply the cumulative distribution function to the intensity corresponding to the pixel and use the result as the new intensity of this pixel. The cumulative distribution function is simply the sum of the probabilities of all the intensities smaller or equal as the intensity of the pixel.

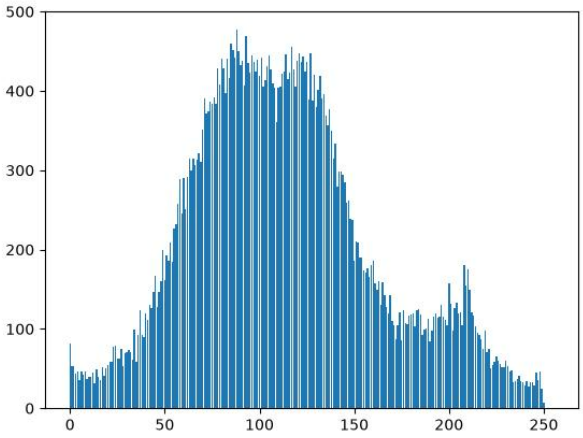
```
@cuda.jit
def equalize(src, hist, dst):
    y = cuda.threadIdx.y + cuda.blockIdx.y * cuda.blockDim.y
    x = cuda.threadIdx.x + cuda.blockIdx.x * cuda.blockDim.x
    if y > src.shape[0] or x > src.shape[1]:
        return

    c = 0
    for i in range(src[y, x]):
        c += hist[i] / pixelCount

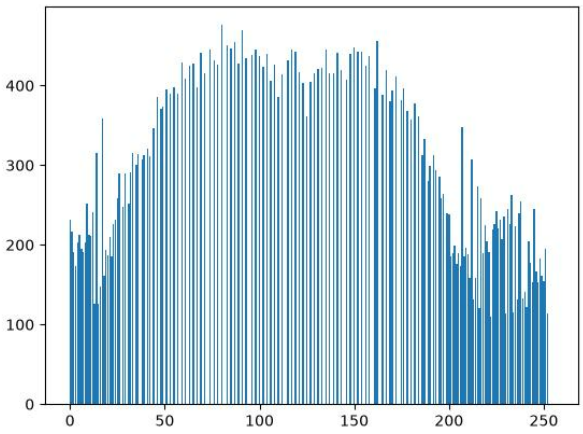
    dst[y, x] = c * 255
```

0.3 Results




500
400
300
200
100
0
0
50
100
150
200
250




400
300
200
100
0
0
50
100
150
200
250